OpenMP — loops, false sharing, and thread affinity

LectureNotes — HPC1 - Università di Trieste, A.Y. 2025-2026
Luca Tornatore, luca.tornatore@inaf.it

16 April 2026. These notes cover the second part of the lecture; the first part — the performance impact of the various synchronisation mechanisms (`critical`, `atomic`, local accumulators) on the `pi_greek example — is the subject of a dedicated, standalone note to which I refer once and for all here.
This document picks up from the moment we start discussing loops, moves to false sharing, and closes on a fundamental topic in most OpenMP introductions: where, physically, our threads are running, and why it matters.

Disclaimer: all the codes, the accompanying plots, the technical explanations and the concept are from the Author. The text has been re-assembled from sparse materials and transcripts of lectures, with a detailed prompt by Claude, and revised by the Author.

OpenMP — loops, false sharing, and thread affinity
- 1. Work-sharing in a loop
  - Two syntactic forms — and the subtle difference
  - The clauses of `#pragma omp for`
- 2. The `reduction` clause
- 3. Assign work to the threads via the `schedule` clause
  - static
  - dynamic
  - guided
  - auto and runtime
  - `04_scheduling.c` — the four regimes
- 4. Loop-specific clauses: `collapse`, `nowait`, `ordered`
  - `collapse(n)`
  - `nowait`
  - `ordered`
- 5. Two things that go wrong: data races and false sharing
  - Data race
  - False sharing
- 6. A scheduling anecdote: when the problem is not in the scheduling
- 7. Where is my thread actually running? NUMA and affinity
  - The machine, from the outside
  - The machine, from inside the node — `lscpu` as a first tool
  - Hardware threads vs. software threads, and what "affinity" means
  - Places: the unit of placement
  - Binding: which place does each thread go to?
  - The trade-off
  - Introspecting from inside the code
  - Examples on the four-socket node
  - First-touch and the other half of the NUMA story
- 8. Remind — red flags to scan for in your own code
- Note: how to limit the threads to specific sets of cores

1. Work-sharing in a loop

Let us start from the most basic question once we have a parallel region and a `for` loop inside it: **how is the iteration space split among the threads?**
Let's suppose we have an iteration space of `N` iterations and a pool of `nthreads` threads. The most natural, hand-made thing we can do is split `N` into roughly equal chunks and give one chunk to each thread. If `N` is not a multiple of `nthreads`, we have a remainder `mod = N % nthreads` to dispose of; the standard trick is to give one extra iteration to each of the first `mod` threads. You find this exact pattern in `loop_without_for.c`:

```
int chunk    = N / nthreads;
int mod      = N % nthreads;
int my_first = chunk*me + ( (me < mod) ? me : mod );
int my_chunk = chunk + (mod > 0)*(me < mod);
```

A very simple pattern: each thread can now run its own loop from `my_first` to `my_first + my_chunk`. Notice that we are *not* using `#pragma omp for` yet — we are doing the work-sharing by hand inside a plain `#pragma omp parallel`. It works.

However, OpenMP offers a specific `#pragma omp for`, for good reasons.

First, *legibility and incremental approach*: historically, OpenMP was born (late 1990s, early 2000s) specifically to parallelise scientific for-loops, and the directive is there to let you write the parallel version with essentially one extra line. Second, and more important: OpenMP lets you choose the **policy** with which the iteration space is cut and assigned, and will implement it for you — something you would need to reimplement by hand every time.

Two syntactic forms — and the subtle difference

There are two equivalent-looking ways to parallelise a loop. They are **not** equivalent:

```
// Form A
#pragma omp parallel
{
    // some work here
    #pragma omp for
    for ( int i = 0; i < N; i++ ) { ... }
    // more work here
}
```

```
// Form B
// not inside a parallel region
#pragma omp parallel for
for ( int i = 0; i < N; i++ ) { ... }
```

Form A opens a parallel region once and uses `omp for` as a *work-sharing* construct inside it; the iteration space is split among the threads of the already-running team. Form B is a convenience shortcut: it opens a parallel region *that coincides* with the for loop, runs the loop in worksharing mode, and closes the region.

If you write `#pragma omp parallel for` **inside** an already-open parallel region, you get *nested parallelism*: every thread of the outer region opens its own parallel region on that loop. If nested parallelism is enabled you will spawn `N_outer × N_inner` threads; if it isn't (the default), each inner region runs with one thread and your loop will not be parallelised at all.

The clauses of `#pragma omp for`

The complete list of clauses for the worksharing form is:

```
#pragma omp for
    schedule( policy [,chunk] )
    ordered
    private       ( var list )
    firstprivate  ( var list )
    lastprivate   ( var list )
    shared        ( var list )
    reduction     ( op: var list )
    collapse      ( n )
    nowait
```

The data-attribute clauses (`private`, `firstprivate`, `lastprivate`, `shared`) I already covered in the parallel-regions note, and they behave exactly the same here. The `reduction` clause was introduced in the `pi_greek` discussion, we'll see it thoroughly later. The genuinely loop-specific ones are `schedule`, `collapse`, `nowait` and `ordered`.

`lastprivate` is specific to worksharing constructs (`for`, `sections`, `taskloop`) and propagates to the outer shared variable the value computed in the *lexically last* iteration — not the one that finishes last in wall-clock time, the one with the largest `i`. The example is in `05_first_and_last_private.c`.

---

2. The `reduction` clause

The `reduction` clause is so ubiquitous in practice that it deserves some attention.

A reduction is a many-to-one operation: you have a pool of threads each producing a partial result, and at the end you want a single scalar — a sum, a product, a min, a max, a logical OR.
Let me remind you what we built by hand in the `pi_greek` example as the `V2` version: every thread carries a local accumulator on its stack, writes into it without any synchronisation during the hot loop, and only at the end we sum the partial results with a small synchronisation cost proportional to `nthreads`, not to `N`.

`reduction(+ : sum)` does exactly that, for you. The code in `01c_array_sum.c` and `01d_array_sum.c`:

```
double *a, sum = 0;
#pragma omp parallel for reduction(+: sum)
for ( int i = 0; i < N; i++ )
    sum += a[i];
```

compiles, in practice, to the same thing you would write explicitly with a local accumulator and a final atomic. Supported operators are `+`, `*`, `-`, `max`, `min`, `&`, `&&`, `|`, `||`. The initial value of the private copy at the beginning of the loop is the neutral element of the operator — `0` for `+`, `1` for `*`, the minimum of the type for `max`, and so on. The initial value of the *shared* variable is combined with the result of the reduction at the end, *not* used as the starting point inside the loop. If `sum` is `100` before the loop and the loop computes `7`, then `sum` ends up as `107`. This trips people up more often than it should.

When there is a `critical` section around a single scalar update (the V0 pattern of `pi_greek`), `reduction` is what you want instead. The exception is if the operator you need is not one of the supported ones (e.g. a custom operation on a non-trivial data structure); in that case you can declare a user-defined reduction with `declare reduction`, or fall back to the manual local-accumulator pattern.

Final note on scaling. Even the `reduction` version of the loop will *not* scale perfectly — it has parallel overhead, as discussed in the standalone notes, and the array summation itself is memory-bandwidth bound. This is the subject of the previous lecture and I will not repeat it here. Just don't expect that using `reduction` makes the problem bandwidth-unbound; it does not.

---

3. Assign work to the threads via the `schedule` clause

The clause is

```
#pragma omp for schedule( policy, chunk )
```

and `policy` can be `static`, `dynamic`, `guided`, `auto` or `runtime`. The `chunk` argument is optional. The policy governs *how* the iteration space is cut into pieces and *how* those pieces are assigned to threads.

static

With `schedule(static, chunk)` the iteration space is cut into chunks of the requested size, and these chunks are assigned to threads in a **predictable, round-robin** fashion. Thread 0 gets chunk 0, thread 1 gets chunk 1, ..., thread `nthreads-1` gets chunk `nthreads-1`, then thread 0 gets chunk `nthreads`, and so on.

If you write `schedule(static)` *without* a chunk size, the iteration space is cut into exactly `nthreads` chunks, one per thread, of size `N/nthreads` (plus the remainder distributed among the first threads, as we did by hand in `loop_without_for.c`). If you write `#pragma omp for` with no `schedule` clause at all, the behaviour is formally *implementation-defined* (the standard says the default is controlled by the internal `def-sched-var` and exposed via `OMP_SCHEDULE`); in practice, on every implementation I have ever used (GCC, LLVM, Intel, NVIDIA HPC SDK) the default is `static` with one chunk per thread. Fine — but don't rely on it being mandated by the standard.

When to use static. Static is good when every iteration carries the same computational cost. The assignment is decided at compile-time-plus-entry, with essentially zero scheduling overhead at runtime, and load balance is perfect by construction.

When static performs poorly. Obviously, static is bad when iterations have unequal cost. A small example — see `04_scheduling.c` — is the following: take 8 iterations with monotonically increasing cost, and 4 threads. Static with chunk 1 assigns iteration 0, 1, 2, 3 to threads 0, 1, 2, 3, and then iterations 4, 5, 6, 7 again to threads 0, 1, 2, 3. Thread 3 gets the two heaviest iterations; thread 0 gets the two lightest. Thread 3 finishes last, and

the others wait for it at the barrier. The golden rule of parallelism — *you can never be faster than your slowest worker* — bites.

dynamic

With `schedule(dynamic, chunk)`, chunks are not pre-assigned. They go into a queue, and whenever a thread finishes the chunk it has been working on, it requests the next free chunk from the queue.
First-come, first-served.

The default chunk size for `dynamic` is **1**, i.e. one iteration per request. Tthe whole *point* of `dynamic` is that fine-grained chunks can actively balance an unpredictable workload, so the choice of chunk size is a tradeoff between the overhead of the queueing and the quality of the load balance.

When to use dynamic. When iteration cost is unpredictable or data-dependent (e.g. ray tracing, tree descent, adaptive mesh work, anything involving an `if` inside the loop body that cuts most iterations short).

When dynamic isn't good. When iterations are uniform: you pay the queueing overhead for no reason. The overhead is modest for reasonable chunk sizes, but it is not zero. However, on modern implementation, the difference could be really unnoticeable.

Remind — an important technical point that is *not* obvious: the OpenMP standard does not specify how `dynamic` must be implemented. In practice it is a lock-protected counter or a work-stealing queue, and that has a cost. On many thousands of short iterations you can actually see the queue become the bottleneck. It is yet another reason to think about the chunk size.

guided

`guided` is a hybrid. The implementation starts with large chunks and shrinks them as fewer iterations remain: typically the next chunk size is proportional to *remaining-iterations / nthreads*, bounded below by the `chunk` argument (default 1). The rationale is to keep the overhead low at the beginning of the loop (big chunks) and retain good load balance at the tail (small chunks).

auto and runtime

`schedule(auto)` lets the implementation choose — useless, in practice, because you don't know what you got. `schedule(runtime)` defers the decision to the runtime, where the policy is read from `OMP_SCHEDULE`.
This latter one is actually useful when you want to benchmark different policies on the same binary; you recompile once and then

```
OMP_SCHEDULE="static"             ./a.out
OMP_SCHEDULE="dynamic,16"         ./a.out
OMP_SCHEDULE="guided"             ./a.out
```

and compare the plots. I recommend it as a debugging tool, not as a production choice.

`04_scheduling.c` — the four regimes

The code tests all three scheduling policies (`static`, `dynamic`, `guided`) on four different workloads:

1. **constant** — every iteration calls `heavy_work(1000)`;
2. **increasing** — iteration `i` calls `heavy_work(K + i)`;

3. **decreasing** — iteration `i` calls `heavy_work(K + N - i + 1)`;
4. **random** — iteration `i` calls `heavy_work(K + rand_r(...) % 2345)`.

What you should expect to see — and what you *will* see if you run it on Leonardo, on your laptop, on one of our Orfeo nodes:

- **constant workload**: the three policies perform within a few percent of each other. If anything, `static` is slightly faster because its overhead is the smallest. Do not read this as "scheduling does not matter" — read it as "when the workload is uniform, the cheap policy is the right one".
- **increasing / decreasing workload**: `static` (with no chunk, so one big chunk per thread) becomes dramatically worse than the other two, because one specific thread gets all the heavy iterations. `dynamic` and `guided` are both good.
- **random workload**: `dynamic` tends to win slightly, because it reacts to the actual distribution rather than hoping for it. The difference is usually smaller than in the increasing case.

Two things worth stressing. First, this is a microbenchmark: do not extrapolate the absolute numbers to your own code, extrapolate the *qualitative behaviour*. Second, the cost of a wrong scheduling choice is multiplicative in `nthreads`: on 4 threads a bad static is mildly annoying, on 64 threads a bad static is a disaster. As the core count grows, the scheduling decision becomes progressively more consequential — not less.

---

4. Loop-specific clauses: `collapse`, `nowait`, `ordered`

`collapse(n)`

When you have nested loops,

```
#pragma omp for collapse(2)
for ( int i = 0; i < Nrows; i++ )
    for ( int j = 0; j < Ncols; j++ )
        A[i][j] = B[i][j] * C[i][j];
```

`collapse(2)` tells OpenMP to linearise the two iteration spaces into a single one of size `Nrows × Ncols`, and then apply the work-sharing to that linearised space. Without it, the worksharing would apply only to the outer loop — which is fine if `Nrows >= nthreads`, but wasteful if `Nrows` is small and the work is actually inside the inner loop.

Two catches.

First, the nested loops **must be perfectly nested** — no code between the two `for` statements, no conditional wrapping the inner loop, nothing. The moment you write

```
#pragma omp for collapse(2)
for ( int i = 0; i < Nrows; i++ ) {
    D[i] = function_of_(i);       // this breaks the nesting
    for ( int j = 0; j < Ncols; j++ )
        A[i][j] = B[i][j] * C[i][j] + D[i];
}
```

`collapse(2)` will not compile (or will silently do something wrong, depending on the compiler — GCC refuses, most others warn or refuse). Hoist `D[i] = function_of_(i)` into a separate outer loop and you are fine.

Second, and more important: `collapse` is a convenience, not a magic speedup. If your two-level nested loop is on an *array-of-pointers* layout (`A` is `double**`, with `Nrows` separate allocations, one per row), `collapse` will cheerfully parallelise it for you — and the result will run badly, because the memory traversal pattern is terrible. The right fix in that case is to allocate `A` as a single contiguous block of `Nrows*Ncols` doubles and address it as `A[i*Ncols + j]`. We discussed this in the optimisation part of the course (matrix transpose, pointer chasing); I will not rehash it here, but: whenever you use `collapse` spend five minutes asking yourself whether the data layout is right in the first place. Especially if the code is ever going to run on a GPU, where pointer chasing is not merely suboptimal — it is catastrophic.

`nowait`

At the end of a work-sharing construct (`for`, `sections`, `single`) OpenMP inserts, by default, an **implicit barrier and a memory flush**. All threads wait for the last one to finish, and the shared memory is synchronised. This is the safe, correct default.

The `nowait` clause removes that barrier:

```
#pragma omp parallel
{
    #pragma omp for nowait
    for ( int i = 0; i < N; i++ )
        a[i] = compute_a(i);

    #pragma omp for
    for ( int i = 0; i < M; i++ )
        b[i] = compute_b(i);
}
```

Here it is correct: the two loops touch different arrays and have no data dependency, so a thread that finishes its share of the first loop can start immediately on the second loop without waiting for anyone. The implicit barrier at the *end of the parallel region* still ensures that `a[]` and `b[]` are consistent before we return to the serial part.

Where `nowait` is wrong:

```
#pragma omp parallel
{
    #pragma omp for nowait
    for ( int i = 0; i < N; i++ )
        a[i] = compute_a(i);

    #pragma omp for
    for ( int i = 0; i < N; i++ )
        b[i] = a[i] * 2;              // <-- reads a[]
}
```

A thread that finished early in the first loop will enter the second loop and read `a[i]` values that other threads have not yet written. Classic data race.

The rule is annoying only because the cost is hidden and the benefit is invisible. As a default, **do not use `nowait`**. Use it only when you have explicitly verified that the loop that follows does not read anything that the previous loop writes. It is one of the top three sources of subtle non-deterministic bugs in OpenMP code.

`ordered`

The `ordered` clause — together with the `ordered` directive inside the loop body — forces the sequential ordering of a section of the loop body. It is useful in a handful of cases (ordered output, certain producer-consumer patterns) and mostly harmful otherwise, because it essentially serialises the ordered region. The `01_where_I_am_omp.c` example uses `ordered` for exactly the right purpose: producing non-interleaved output lines in thread-number order. Outside of that kind of I/O-ordering job, if you think you need `ordered`, think again first.

---

5. Two things that go wrong: data races and false sharing

Parallel loops expose two classes of bug that deserve to be named distinctly, because one is a *semantic* bug and the other is a *performance* bug, and confusing them is how people end up shouting at OpenMP for the wrong reasons.

Data race

A **data race** is the condition in which:

1. two or more threads access the **same** memory location,
2. **concurrently** (not separated by a synchronisation point),
3. at least one of the accesses is a **write**,
4. the accesses are **not synchronisation operations** themselves (i.e. they are not atomics, they are not locks).

If all four conditions hold, the behaviour is formally undefined. This is a *semantic* error: the program is broken; the result is non-deterministic; on different runs, with different compilers, or with different core counts you may get different answers. The classical example is our first-attempt reduction,

```
#pragma omp parallel for
for ( int i = 0; i < N; i++ )
    sum += a[i];
```

where `sum += a[i]` expands (on x86, unoptimised) into a load into a register, an add, and a store back — three instructions, not one atomic operation. Two threads can interleave their loads and stores in an order that loses updates. The slide with the cycle-by-cycle table in `03loops.pdf` shows one such pathological interleaving.

Fixes: `reduction`, local accumulators, `atomic`, `critical`. In this specific case: `reduction`, every single time.

False sharing

A **false sharing** situation, on the other hand, is:

1. each thread accesses a memory location that is **different** from any other thread's,
2. **but** at least some of those locations lie within the same **cache line**,
3. and at least some accesses are writes.

This is **not a data race**. The code is semantically correct; the result is deterministic and right. But the performance is pretty poor.

Why? Because caches work by *lines*, not bytes. A typical x86-64 cache line is 64 bytes. If thread 0 writes to `sum[0]` and thread 1 writes to `sum[1]`, and `sum[0]` and `sum[1]` live in the same line, then the MESI (or MESIF, or MOESI — depending on the vendor) cache-coherence protocol will invalidate the line in thread 1's L1 every time thread 0 writes, and vice versa. The line ends up ping-ponging between the two L1 caches — or, across sockets, across the UPI / Infinity Fabric interconnect — once per iteration. The cost is two or three orders of magnitude worse than a hot local write.

The canonical bait is the "local accumulator in a shared array" pattern:

```
double sum[nthreads];          // one slot per thread — shared array
#pragma omp parallel
{
    int me = omp_get_thread_num();
    #pragma omp for
    for ( int i = 0; i < N; i++ )
        sum[me] += a[i];        // no data race: each thread writes its own slot
}
```

No data race — each thread writes only `sum[me]`, no one else reads or writes it inside the loop. But `sum[0]`, `sum[1]`, ..., `sum[7]` occupy 64 contiguous bytes: one single cache line. Every write by thread `me` invalidates the line for everybody else. This is `02_falsesharing.c`.

The cure — when a genuine per-thread variable has to be accessible from outside the parallel region (so you cannot just put it on the thread's stack) — is **padding**. You give each thread enough space that its slot falls on its own cache line. The ugly-but-portable pattern is

```
double S[nthreads][8];     // 8 doubles = 64 bytes = one full line, per thread
...
#pragma omp for
for ( int ii = 0; ii < N; ii++ )
    S[me][0] += array[ii];
```

This is `03_falsesharing_fixed.c`. You waste 56 bytes per thread; in exchange, you buy back your scaling. Honest trade. For more serious code you can use `alignas(64)` (C11) or the OpenMP 5.x `aligned` clause, but the `[nthreads][8]` pattern is old and works on everything.

Two final remarks:

1. The same pattern applies to **structures**: if an array of structures is accessed by different threads based on the element index, and the structure is small, two threads may easily touch different elements that

share a cache line. Padding the structure to a cache-line boundary cures it. The cost is memory; the gain is scaling.
2. False sharing is, in my experience, the single most common hidden cause of "my OpenMP code does not scale" in legacy scientific codes. It is not a data race — no correctness tool will flag it. It is not an algorithmic flaw — your algorithm on the blackboard still scales perfectly. It is an implementation-level interaction with the cache-coherence protocol, and the only way to catch it in production is either to profile (L3 invalidations, HITM events) or to *know the pattern* and look for it. Know the pattern.

---

6. A scheduling anecdote: when the problem is not in the scheduling

If you run a *perfectly balanced* `reduction`-based array summation across many threads, you will find — especially on a two-socket or four-socket node — that the parallel efficiency drops well below 1 even though we did everything right. No data race, no false sharing, no critical region, reduction as it should be. And yet the scaling is mediocre.

What gives? The answer is not in the scheduling. The answer is in *where the data are* — relative to the thread that is summing it. If the array was allocated before the parallel region by thread 0, then all of it ends up on thread 0's NUMA node (because of the first-touch policy, which we will explain in a minute). Every other thread that now wants to read a chunk of it is reading from *remote* memory, over the inter-socket interconnect. Bandwidth per thread collapses.

This is the bridge to the second half of the lecture. The scheduling policy decides how *iterations* are distributed among threads; it has no say in where the *memory* is, nor where the threads themselves are. And for a memory-bound loop, those two things dominate performance.

When you see unexplained poor scaling on a `reduction`-style loop, *do not blame the scheduler*. Blame memory placement first.

---

7. Where is my thread actually running? NUMA and affinity

This is the part of the lecture that was cut from the recording due to a technical issue. I reconstruct it here from the slides and from what I normally say in class; if something reads as terser than the rest of the note, that's why.

The machine, from the outside

A modern server node typically has multiple sockets (2, 4, sometimes 8). Each socket is a package: inside, many cores (12, 24, 36, 64, 96, ...), each with private L1 and L2, sharing a last-level L3 that is internal to the socket. Each socket is physically connected to one or more memory banks — *its* memory — via its own set of memory controllers. Sockets are then interconnected with some technology: Intel's UPI (formerly QPI), AMD's Infinity Fabric, various things on ARM.

Crucial consequence: all the memory on the node is addressable by every core (that is what "shared memory" means), but **access to memory attached to another socket is slower than access to local memory**. This is the meaning of NUMA — *Non-Uniform Memory Access*. On a 4-socket Intel node, a typical pattern of `node distances` (from `lscpu` or `numactl --hardware`) looks like:

```
node   0   1   2   3
   0: 10  21  21  21
   1: 21  10  21  21
   2: 21  21  10  21
   3: 21  21  21  10
```

The numbers are *relative*, not physical units. `10` is the cost of accessing your own memory; `21` is the cost of accessing memory on another socket, meaning roughly 2× slower. The fact that the off-diagonal is *uniformly* 21 — even though the physical topology has some pairs of sockets directly connected by UPI and others reachable only through a hop — is partly an Intel simplification exposed to userland; on AMD Epyc systems and on more recent platforms you often see a finer matrix, with different values depending on the specific socket pair, and sometimes with sub-socket NUMA domains (the "NPS4" BIOS setting on Epyc splits each socket into 4 NUMA nodes, reflecting the real CCD organisation). When in doubt, trust the matrix your system reports over your topological expectations.

The machine, from inside the node — `lscpu` as a first tool

Before you place threads, know the machine you are on. The usual first command is `lscpu`. On one of our four-socket nodes (the same one used for the pi_grid experiments), it reports:

- `Thread(s) per core: 2` — SMT/Hyper-Threading is on.
- `Core(s) per socket: 12`
- `Socket(s): 4`
- `NUMA node(s): 4` — one per socket.
- `CPU(s): 96` = 4 × 12 × 2 — the total number of *logical* processors, which the kernel and OpenMP call "CPUs" even though they are not sockets in the hardware sense. (The HPC jargon is genuinely confusing: *CPU = socket* in common speech, *CPU = logical processor / hardware thread* in `lscpu` / `taskset` / `numactl` output. Get used to flipping between the two.)
- `NUMA node0 CPU(s): 0-11,48-59`, `NUMA node1 CPU(s): 12-23,60-71`, ..., which tells you that logical processors 0–11 are the physical cores of socket 0, and 48–59 are the sibling SMT threads of those same cores. Round-robin numbering of the SMT sibling starts from 48 because SMT siblings are enumerated *after* all the physical cores have been enumerated.

Other useful tools: `numactl --hardware`, `lstopo` / `hwloc-ls` (from OpenMPI — arguably the cleanest visual), Intel's `cpuinfo`. Use them. Don't assume the machine topology from the documentation, check it. I have seen too many people run an OpenMP benchmark on a node thinking they had 64 cores when they actually had 32 cores with SMT.

Hardware threads vs. software threads, and what "affinity" means

We need two pieces of vocabulary:

- A **hardware thread** is a physical execution context — a core, or one of the SMT siblings of a core if SMT is on.
- A **software thread** is what OpenMP spawns — a pthread, in practice.

The mapping between software threads and hardware threads is called **thread affinity**. The operating system is absolutely free, by default, to migrate software threads across hardware threads whenever its scheduler decides so — it usually does this to balance load across the machine, which is great for a general-purpose workstation and *terrible* for an HPC workload, because migration discards your L1/L2 cache and, if you cross a socket, forces everything you have in memory to suddenly become "remote".

The job of the OpenMP affinity mechanism is to tell the OS: *do not migrate my threads, or migrate them only within this or that boundary*, and *place thread* `i` *here, thread* `i+1` *there.* OpenMP exposes this through two concepts: **places** and **binding**.

Places: the unit of placement

A **place** is the smallest hardware entity on which OpenMP will agree to put (and keep) a thread. OpenMP offers three canonical granularities, set with `OMP_PLACES`:

- `OMP_PLACES=sockets` — each place is a whole socket. A thread placed on a socket may, internally, be migrated by the OS between the cores of that socket, but never leaves it.
- `OMP_PLACES=cores` — each place is a physical core. A thread placed on a core may be migrated between the SMT siblings of that core (if SMT is on) but not to another core.
- `OMP_PLACES=threads` — each place is a single hardware thread (a physical core if SMT is off, one SMT sibling if SMT is on). A thread placed here does not migrate at all.

Finer than "threads" you cannot get; this is the hardware floor.

You can also give an explicit list of places, using a sub-list syntax:

```
OMP_PLACES="{0,1},{2,3},{4,5},{6,7}"
```

defines four places, each made of two hardware threads. The numbers are the "CPU IDs" in the kernel sense — the ones that appear in `NUMA node0 CPU(s): 0-11,48-59` etc. This syntax is useful when you want to carve out a specific subset of the machine (for instance, all physical cores of socket 1, but not their SMT siblings: `OMP_PLACES="{12,13,14,15,16,17,18,19,20,21,22,23}"`).

Binding: which place does each thread go to?

Once the set of places is defined, the *binding policy* — set with `OMP_PROC_BIND` — decides how OpenMP distributes its `nthreads` software threads across those places. The policies are:

- `OMP_PROC_BIND=false` — no binding. The OS is free to migrate. Don't use this in production.
- `OMP_PROC_BIND=true` — bind, but do not specify the pattern. The implementation chooses. Don't use this either; you don't know what you got.
- `OMP_PROC_BIND=close` — place threads **as close as possible** in the place list. Thread 0 on place 0, thread 1 on place 1, ..., thread `k` on place `k mod nplaces`. "Close" is with respect to the *ordering of the place list*, which typically corresponds to hardware adjacency.
- `OMP_PROC_BIND=spread` — place threads **as spread out as possible** across the place list. If you have 4 places and 2 threads, `spread` gives you thread 0 on place 0 and thread 1 on place `nplaces/2` (= place 2), maximising the distance.
- `OMP_PROC_BIND=master` (alias `primary` in OpenMP ≥ 5.0) — all threads go to the same place as thread

0. This is almost never what you want in production, but it is a useful diagnostic: you can force massive oversubscription of a single place and see what happens to your code, or constrain a team to the master's NUMA node for a small region.

You can also pass a **comma-separated list** of policies to control nested parallelism:

```
OMP_PROC_BIND=spread,close
```

meaning: at the outer level, spread; at the inner level, close. This is the right setting for many nested OpenMP codes: spread the outermost threads across sockets to maximise aggregate bandwidth, then pack the inner team close together to share the L3.

The trade-off

There is no universal "best" binding. The choice is a genuine trade-off.

- **Packing threads close together** (same socket, or even same core's SMT siblings):
  - *Pros*: synchronisation is cheap (cache-coherence traffic stays inside the L3 of one socket; you never go through the inter-socket interconnect for a barrier). Threads that share data benefit from a warm L3.
  - *Cons*: you are sharing the memory bandwidth of one memory controller among many threads; for a bandwidth-bound loop, aggregate bandwidth is determined by the single socket's memory bandwidth, which is halved (or quartered) compared to using all sockets.
- **Spreading threads across sockets**:
  - *Pros*: each socket has its own memory controller; aggregate bandwidth scales with the number of sockets. Each thread has essentially an entire L3 to itself.
  - *Cons*: synchronisation crosses the interconnect and is much more expensive. The `pi_grid` slowdowns (1000× on two sockets, 2000× on four sockets) were this exact effect: a critical region that requires a handoff on every iteration becomes dramatically more expensive when the handoff has to traverse UPI.

The reasonable rule of thumb:

- **Bandwidth-bound code, little synchronisation** → spread. You want aggregate bandwidth.
- **Synchronisation-heavy code, small working set** → close. You want the cache-coherence traffic to stay local.
- **Mixed workloads, or codes that do both sorts of thing** → there is no silver bullet; either pick the most performance-critical phase and bind for it, or switch the binding per region using the `proc_bind(...)` clause on `#pragma omp parallel`. And profile — don't just guess.

I should be candid: picking the binding is one of the few places in HPC where I tell people to *measure* before reasoning. The theoretical arguments are usually right in their sign, but the magnitudes vary by one or two orders of magnitude between platforms and even between generations of the same vendor. `OMP_PLACES` and `OMP_PROC_BIND` are cheap to change; rerunning a scalability test is cheap. Do it.

Introspecting from inside the code

It is one thing to set `OMP_PLACES` and `OMP_PROC_BIND` in the shell; it is another to verify that the runtime actually honoured you. The OpenMP runtime API exposes a handful of introspection calls that tell you exactly where each software thread is:

- `omp_get_place_num()` — the index (in the place list) of the place this thread is currently bound to.
- `omp_get_num_places()` — total number of places.
- `omp_get_place_num_procs(place)` — how many hardware threads make up `place`.
- `omp_get_place_proc_ids(place, ids)` — the kernel-side CPU IDs forming that place.
- `omp_get_proc_bind()` — the current binding policy.
- `omp_get_partition_num_places()` — the number of places available to the current team (useful in nested scenarios).

The example `01_where_I_am_omp.c` does exactly this walk and prints, for each thread, the place it sits on and the list of hardware threads that place contains. A sibling example, `00_where_I_am.c`, goes one level deeper: it asks the kernel directly *on which hardware thread* the calling thread is running, using either `sched_getcpu()` (GNU glibc), the `SYS_getcpu` system call, or — as a fallback — field 39 of `/proc/self/stat`. The fallback is not pretty but it is instructive: it illustrates that everything you see in userspace about process state is ultimately exposed by the kernel through `/proc`, and if you lack the fancy wrappers you can always parse the `/proc` entry yourself. (Sources: Linux kernel documentation on `/proc/<pid>/stat`, specifically the `task_cpu` field at position 39 in the canonical field ordering; do note that this ordering has been stable for a very long time, but the kernel docs don't formally guarantee it won't change. For production code prefer `sched_getcpu()`.)

The two codes together give you a cheap sanity check: set `OMP_PLACES`, set `OMP_PROC_BIND`, run `./01_where_I_am_omp N`, and read the output. If threads are not where you think they should be, something is off — either your variables are not getting picked up (common issue: you exported them in a shell that the launcher does not inherit from), or a higher-priority mechanism (SLURM's `--cpu-bind`, a container runtime, a wrapping script) is overriding them. Sort that out before you even start the scalability study.

Examples on the four-socket node

To make the abstract concrete, on our 4-socket node with 12 cores per socket and SMT on (96 logical CPUs total):

- `OMP_PLACES=threads OMP_PROC_BIND=close` with 4 threads → threads 0, 1, 2, 3 all end up on the first four logical CPUs of socket 0: cores 0, 1, 2, 3 (physical threads, not yet SMT siblings). Everything is packed on one socket.
- `OMP_PLACES=threads OMP_PROC_BIND=spread` with 4 threads → threads 0, 1, 2, 3 end up one per socket, approximately on the first logical CPU of each.
- `OMP_PLACES=threads OMP_PROC_BIND=master` with 96 threads → all 96 threads forced onto the single hardware thread of core 0. Nothing runs in parallel in any useful sense; this is only ever useful as a test of "what does my code do under pathological oversubscription?".
- `OMP_PLACES=cores OMP_PROC_BIND=close` with 8 threads → threads 0 through 7 on cores 0 through 7 of socket 0, SMT siblings not used.

- `OMP_PLACES=cores OMP_PROC_BIND=spread` with 8 threads → threads distributed two per socket (8 threads / 4 sockets), again on physical cores with SMT siblings left alone.

The slides include a large matrix of such examples for different `(places, binding, nthreads)` combinations on this specific node; I do not reproduce it here because the point is not to memorise the matrix but to reproduce it yourself on the machines you actually use. The takeaway is simply that the two environment variables, together, determine the answer completely and predictably — there are no hidden parameters.

First-touch and the other half of the NUMA story

I am closing on affinity, but I want to flag explicitly — because it was lost in the recording cut — that *thread affinity is only half of the NUMA story*. The other half is **memory placement**: where the pages your threads access are physically located. On Linux the default policy is **first-touch**: a page is physically allocated on the NUMA node of the thread that first writes to it (not the one that `malloc()`'d or `calloc()`'d it — the kernel only reserves the virtual range on `malloc`, the page only gets physically backed on the first access).

This has a very practical consequence for OpenMP code: *if you allocate and initialise a large array serially before the parallel region, the whole array ends up on the NUMA node of thread 0*. Every subsequent parallel access from threads on other sockets is a remote access. The cure is to do the initialisation **in parallel**, with the same work-distribution pattern that the computation loop will use. If your computation does

```
#pragma omp parallel for schedule(static)
for ( int i = 0; i < N; i++ )
    y[i] = alpha * x[i] + y[i];
```

then the initialisation should also be

```
#pragma omp parallel for schedule(static)
for ( int i = 0; i < N; i++ ) {
    x[i] = ...;
    y[i] = ...;
}
```

— same schedule, same binding. This ensures first-touch places each page on the NUMA node of the thread that will later consume it. The effect on performance, for bandwidth-bound kernels on multi-socket nodes, is often a factor of 2× or more.

This is a deep topic and I am compressing it badly in one paragraph. It deserves — and will get — a dedicated note of its own. But the one-liner to take away today is: **`OMP_PLACES` / `OMP_PROC_BIND` alone are not NUMA-awareness. They place your threads; they do not place your data. Pair them with parallel first-touch initialisation.**

---

8. Remind — red flags to scan for in your own code

A short checklist from the whole lecture, for when you are reviewing OpenMP code that doesn't scale. If any of these is true, stop and fix it before doing anything else:

- a `#pragma omp critical` or `#pragma omp atomic` inside a hot loop body;
- a `#pragma omp parallel for` written inside an already-open parallel region (nested parallelism you

did not intend);

- a loop with visibly non-uniform iteration cost under `schedule(static)` with no chunk size;
- a per-thread array (`double sum[nthreads]`) used as accumulator without padding — false sharing waiting to happen;
- a `nowait` on a loop whose output is read by the next loop;
- `collapse` on a pointer-chased 2D layout (`double **A`);
- a scalability study without a specified `OMP_PROC_BIND` — you are measuring the OS scheduler's mood, not your code;
- a benchmark where the initialisation is serial and the computation is parallel — you are measuring remote-memory accesses, not your algorithm.

None of these is a theoretical concern. Each of them has cost me a day of debugging at least once.

Cross-references to example codes used in this lecture:

- `loop_without_for.c` — manual chunking, no `omp for`.
- `00_array_sum_with_race.c` — the broken baseline (data race).
- `01a..01d_array_sum.c` — successive refinements: `critical`, `atomic`, local accumulators, `reduction`.
- `02_falsesharing.c` / `03_falsesharing_fixed.c` — anatomy of false sharing and the padding cure.
- `04_scheduling.c` — the three scheduling policies under four workload shapes.
- `05_first_and_last_private.c` — `firstprivate` and `lastprivate` semantics.
- `00_where_I_am.c` — low-level introspection via `sched_getcpu()` / `/proc/self/stat`.
- `01_where_I_am_omp.c` — the OpenMP runtime API for places, binding and placement introspection.

Companion notes:

- the standalone note on the `pi_grid` example covers the synchronisation-overhead part of this lecture in depth and is not repeated here;
- a dedicated note on NUMA awareness and first-touch memory placement will follow; the present document only sketches the problem.

Note: how to limit the threads to specific sets of cores

Remind that you can limit the sets of cores available for your threads, or in general you can pin a process to run on a specific subset of the available cores; for instance by using `taskset`:
`taskset -c *list-of-cores*`
see the `man` page for details.